

# Machine Learning Notes

## Preliminary Introduction to Deep Learning

Vitor Negromonte Cabral de Oliveira

December 2024

### **Abstract**

This notebook serves as a comprehensive repository for my Machine Learning studies, with a primary focus on Deep Learning. Key topics covered include:

- Artificial Neural Networks (ANNs) – Fundamentals and architectures with mathematical explanation.



# 1 Introduction to Artificial Neural Networks

## 1.1 Perceptron

Nearly every modern artificial neural network, from simple classifiers to sprawling deep learning architectures, traces its origins to a simple computational unit called the **perceptron**<sup>1</sup>.

Developed by Frank Rosenblatt in 1947, inspired by the work of McCulloch and Pitts in 1943. The perceptron, sometimes called the *sigmoid neuron*, it's traced back from the mathematical interpretation of the biological neurons' way of functioning.

At its core, the perceptron is a binary decision-maker. It takes a set of real-valued inputs, weighs their importance, and makes a crisp, threshold-based choice. Imagine a perceptron as a tiny gatekeeper: it evaluates evidence (inputs), applies its own biases and preferences (weights), and delivers a verdict — either 1 ("yes") or 0 ("no").

### 1.1.1 Mathematical Foundation of Perceptrons

Given inputs  $x_1, x_2, \dots, x_n$  the perceptron's output ( $X$ ) is governed by two steps:

1. **Weighted Sum:** Compute a linear combination of inputs, adjusted by weights  $w_i$  and a bias term  $w_0$  (which shifts the decision boundary):

$$z = w_0 + \sum_{i=1}^n w_i x_i \quad (1)$$

2. **Thresholding:** Apply a step function to produce the final output:

$$o(x_1, \dots, x_n) = \begin{cases} 1 & \text{if } w_0 + \sum_{i=1}^n w_i x_i > 0 \quad (\text{activation}), \\ 0 & \text{otherwise.} \end{cases} \quad (2)$$

Each weight  $w_i$ , a real-valued constant also called a *parameter*, determines the contribution of input  $x_i$  to the perceptron's output. The bias term  $w_0$  acts as an adjustable threshold: the weighted sum  $\sum_{i=1}^n w_i x_i$  must exceed  $-w_0$  to trigger a +1 output. To simplify notation, we introduce a constant input  $x_0 = 1$ , allowing the activation condition to be written compactly as  $\sum_{i=0}^n w_i x_i > 0$  or, in vector form,  $\mathbf{w} \cdot \mathbf{x} > 0$ .

Proposed by McCulloch and Pitts in 1943, the Perceptron can be used to represent many boolean functions, indeed, it can represent all of the primitive boolean functions (AND, OR, NAND and NOR). Unfortunately, however, some boolean functions can be represented by a single perceptron, such as the XOR function whose value is  $1 \leftrightarrow x_1 \neq x_2$ . This limitation of a single perceptron unit to not be able to represent a XOR function started the first Computational Intelligence Winter.

### 1.1.2 Geometric Interpretation of the Perceptron

The perceptron's decision mechanism admits a geometric interpretation. Consider a perceptron with  $n$  inputs: its weight vector  $\mathbf{w} = (w_0, w_1, \dots, w_n)$  and input vector  $\mathbf{x} = (1, x_1, \dots, x_n)$  define a hyperplane in  $\mathbb{R}^{n+1}$  space:

$$\mathbf{w} \cdot \mathbf{x} = w_0 + \sum_{i=1}^n w_i x_i = 0 \quad (3)$$

This *decision boundary* partitions the input space into two distinct regions:

- $\mathbf{w} \cdot \mathbf{x} > 0$  corresponds to the output +1 (positive class)

---

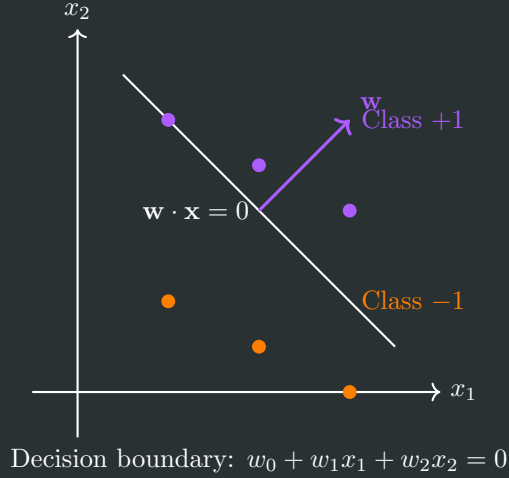
<sup>1</sup>While most neural networks rely on perceptron-like units, alternative paradigms exist, such as Hebbian learning (inspired by synaptic plasticity) and the recently revived Kolmogorov-Arnold Network (KAN), which bypasses traditional neuron structures entirely.

- $\mathbf{w} \cdot \mathbf{x} < 0$  corresponds to the output  $-1$  (negative class)

For the special case of two-dimensional inputs  $(x_1, x_2)$ , this reduces to a line in  $\mathbb{R}^2$ :

$$w_0 + w_1x_1 + w_2x_2 = 0 \quad (4)$$

The bias term  $w_0$  determines the offset from the origin, while the ratio  $w_1/w_2$  gives the slope. The weight vector  $\mathbf{w}$  is always *normal* (perpendicular) to the decision boundary, pointing toward the positive region.



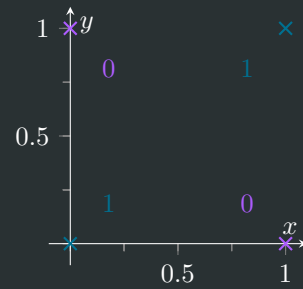
This geometric perspective reveals the perceptron's fundamental limitation: it can only learn *linearly separable* functions. The classic XOR problem exemplifies this constraint - no single line can separate  $(0, 1)$  and  $(1, 0)$  from  $(0, 0)$  and  $(1, 1)$  in  $\mathbb{R}^2$ . This motivated the development of multilayer perceptrons (MLPs) that can construct nonlinear decision boundaries through composition of multiple hyperplanes.

### 1.1.3 The XOR Problem

The "XOR Problem" is a classical problem in Computational Intelligence, proposed initially by Marvin Minsky in his book "Perceptron" (1969). It's the problem of using a Perceptron to predict the output of XOR logic gates given two binary outputs. A XOR function should only return true if the two inputs are not equal and a false if they are equal.

| Input 1 | Input 2 | Output |
|---------|---------|--------|
| 0       | 0       | 0      |
| 1       | 0       | 1      |
| 0       | 1       | 1      |
| 1       | 1       | 0      |

(a) XOR truth table



(b) XOR visualization

Figure 1: XOR operation represented as a truth table (a) and a visualization (b). The plot shows the XOR output for different inputs, where 0s are marked in Orchid and 1s are marked in MidnightBlue.

The structure of a perceptron unit is limited to separating the data point with a single line and since the XOR inputs are not linearly separable. This can be solved by adding an extra layer of neurons to the network, called hidden-layer, this architecture is called Multilayer perceptron (next section).

## 1.2 Gradient Descent and Stochastic Gradient Descent

### 1.2.1 Perceptron Training Rule

One way to learn an acceptable weight vector is to begin with random weights, then iteratively apply the perceptron to each training example, modifying the perceptron weight whenever it misclassifies an example. This process is repeated, iterating through the training examples as many times as needed until the perceptron classifies all training examples correctly. Weights are modified at each step according to the perceptron training rule, which revises the weight  $w_i$  associated with input  $x_i$  according to the rule

$$w_i \leftarrow w_i + \Delta w_i \quad (5)$$

where

$$\Delta w_i = \eta(t - o)x_i \quad (6)$$

And  $t$  stands for the target output for the current training example,  $o$  is the output generated by the perceptron and  $\eta$  is a positive constant called learning rate. The role of the learning rate is to moderate the degree to which weights are changed at each step. It is usually set to some value between 0 and 1, and sometimes made to decay as the number of weight-tuning iterations increases.

### 1.2.2 Delta Rule

Although the perceptron rule finds a successful weight vector when the training examples are linearly separable, it can fail to converge if the examples are not linearly separable. A second training rule, called the delta rule, is designed to overcome this problem. The key idea behind the delta rule is to use gradient descent to search the hypothesis space of possible weight vectors to find the weights that best fit the training examples. This rule sets the basis for the Backpropagation Algorithm. The delta training rule is best understood by considering the task of training an unthresholded perceptron; that is, a linear unit for which the output  $o$  is given by

$$o(\vec{x}) = \vec{w} \cdot \vec{x} \quad (7)$$

Thus, a linear unit corresponds to the first stage of a perceptron, without the threshold. In order to derive a weight-learning rule for linear units, let us begin by specifying a measure for the training error of a hypothesis, relative to the training examples. Although there are many ways to define the error, one common measure that will turn out to be especially convenient is:

$$E(\vec{w}) = \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2 \quad (8)$$

where  $D$  is the set of training examples,  $t_d$  is the target output for training example  $d$ , and  $o_d$  is the output of the linear unit for training example.

### 1.2.3 Gradient Descent

To better understand the Gradient Descent, it's helpful to visualize the entire hypothesis space of possible weight vectors and their associated  $E$  values (Figure ??). The Gradient Descent search determines a weight vector that minimizes  $E$  by starting with an arbitrary initial weight vector, then repeatedly modifying it in small steps. At each step, the weight vector is altered in the direction that produces the steepest descent along the error surface depicted in Figure ??.

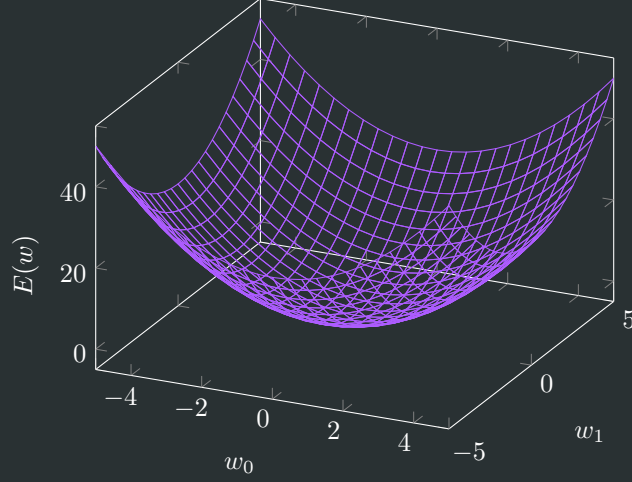


Figure 2: Generic representation of a hypothesis space

Based on that, we can calculate the direction of steepest descent along the error surface ( $E$ ) computing the derivative of  $E$  with respect to each component of the vector  $\vec{w}$ . This vector derivative is called the gradient of  $E$  with respect to  $\vec{w}$ , written  $\nabla E(\vec{w})$ .

$$\nabla E(\vec{w}) \equiv \left[ \frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \dots, \frac{\partial E}{\partial w_n} \right] \quad (9)$$

And  $\nabla E(\vec{w})$  is itself a vector, whose components are the partial derivatives of  $E$  with respect to each of the  $w_i$ . When interpreted as a vector in weight space, the gradient specifies the direction that produces the steepest increase in  $E$ . Since the gradient specifies the direction of steepest increase of  $E$ , the training rule for gradient descent is

$$\vec{w} \leftarrow \vec{w} + \Delta \vec{w} \quad (10)$$

where

$$\Delta \vec{w} = -\eta \nabla E(\vec{w}) \quad (11)$$

The negative sign is present because we want to move the weight vector in the direction that decreases  $E$ . This training rule can also be written in its component form

$$w_i \leftarrow w_i + \Delta w_i \quad (12)$$

where

$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i} \quad (13)$$

To efficiently calculate the gradient at each step, we need to obtain the derivatives of  $\frac{\partial E}{\partial w_i}$  by differentiating

E from Equation ?? as:

$$\begin{aligned}
\frac{\partial E}{\partial w_i} &= \frac{\partial}{\partial w_i} \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2 \\
&= \frac{1}{2} \sum_{d \in D} \frac{\partial}{\partial w_i} (t_d - o_d)^2 \\
&= \frac{1}{2} \sum_{d \in D} 2(t_d - o_d) \frac{\partial}{\partial w_i} (t_d - o_d) \\
&= \sum_{d \in D} (t_d - o_d) \frac{\partial}{\partial w_i} (t_d - \vec{w} \cdot \vec{x}_d) \\
&= \sum_{d \in D} (t_d - o_d) (-x_{id})
\end{aligned} \tag{14}$$

where  $x_{id}$  denotes the single output component  $x_i$  for training example  $d$ . With all that, we now have an equation that gives  $\frac{\partial E}{\partial w_i}$  in terms of the linear unit inputs  $x_{id}$ , outputs  $O_d$ , and target values  $t_d$  associated with the training examples. With this, we have:

$$\Delta w_i = \eta \sum_{d \in D} (t_d - o_d) x_{id} \tag{15}$$

#### 1.2.4 Stochastic Approximation to Gradient Descent

Due to its computation complexity, usually, we do not use directly GD. The need of using a whole set at each iteration during training, makes it more computationally expensive to use GD. Based on that, we do use a stochastic approximation to gradient descent, mostly called stochastic gradient descent or SGD. In SGD, instead of using the entire dataset for each iteration, only a single random sub-set is selected to calculate the gradient and update model parameters. The random selection introduces randomness into the optimization process (stochastic stands for being well-described by a random probability distribution). Mathematically it can be described as

$$w_i := w_i - \eta \nabla Q_i(w) \tag{16}$$

The randomness of the technique can also provoke some noise in the gradient, which some big variations in the gradient.

### 1.3 Multilayer Perceptron and Backpropagation

To address the limitations of a single artificial neuron, which can only represent linear decision boundaries, we explore the concept of a multilayer network. In this approach, multiple fully connected layers are stacked on top of each other, enabling the model to learn and represent more complex, nonlinear decision surfaces.

For learning purposes, we'll address the previous network (perceptron) as a Shallow Neural Network, we call it that way, because the only way to increase the number of neurons it's adding them "*vertically*" due to the nature of a single layer of neurons. More over, in this chapter we'll dive into the evolution of these concepts, from Shallow Neural Networks to Deep Neural Networks and its impacts in the field.

The simple way of making a neural network deeper it's to compositing shallow neural networks. Basically, the output of the first unit becomes the input of the second unit.

In first sight, we need to decide what type of unit shell we use as the basis for constructing multilayer networks. Initially, we might be tempted to choose the linear units discussed before, for which we already derived a gradient descent learning rule. However, multiple layers of cascaded linear units still produce only linear functions. What we need is a unit whose output is a nonlinear function of its inputs, but whose output is also a differentiable function of its inputs, like a sigmoid unit or a tanh unit or a ReLU unit, any function can be use since it covers all the requirements described above.

### 1.3.1 Backpropagation Algorithm

The BP algorithm *learns* the weights for a multilayer network, given a network with a fixed set of units and interconnections. It employs gradient descent to attempt to minimize the squared error between network output values and target values for these outputs. Because we are considering networks with multiple output units rather than single units as before, we begin by redefining  $E$  to sum the errors over all of the network output units:

$$E(\vec{w}) \equiv \frac{1}{2} \sum_{d \in D} \sum_{k \in \text{outputs}} (t_{kd} - o_{kd})^2 \quad (17)$$

where *outputs* is the set of output units in the network, and  $t_{kd}$  and  $o_{kd}$  are the target and output values associated with the  $k$ th output unit and training example  $d$ .

The learning problem faced by Backpropagation is to search a large hypothesis space defined by all possible weight values for all the units in the network, as illustrated in Figure ???. The error in that diagram is replaced by our new definition of  $E$ , and the other dimensions of the space correspond to all the weights associated with all of the units in the network. As in the case of training a single unit, gradient descent can be used to attempt to find a hypothesis to minimize  $E$ .

One major difference in the case of multilayer networks is that the error surface can have multiple local minima, in contrast to the single-minimum parabolic error surface we saw in Figure ???. This means that gradient descent is guaranteed only to converge toward some local minimum, and not necessarily the global minimum error.

The algorithm proceeds as follows:

- Each training example is a pair of the form  $\langle \vec{x}, \vec{t} \rangle$ , where  $\vec{x}$  is the vector of network input values, and  $\vec{t}$  is the vector of target network output values.
- $\eta$  is the learning rate.  $n_{in}$  is the number of network inputs,  $n_{hidden}$  the number of units in the hidden layer, and  $n_{out}$  the number of output units.
- The input from unit  $i$  into unit  $j$  is denoted  $x_{ij}$ , and the weight from unit  $i$  to unit  $j$  is denoted  $w_{ij}$ .

The Backpropagation algorithm involves two phases for each training example:

#### Forward Pass:

1. Initialize the network by assigning random small values to all weights  $w_{ij}$ .
2. Present the input vector  $\vec{x}$  to the input layer.
3. Propagate the activations forward through the network using the activation function  $o_j = f(\sum_i w_{ij}x_{ij})$ , where  $f$  is a differentiable activation function (e.g., sigmoid or ReLU).

#### Backward Pass (Error Computation and Weight Update):

1. Compute the error for the output layer:  $\delta_k = (t_k - o_k)f'(o_k)$ , where  $f'(o_k)$  is the derivative of the activation function.
2. Compute the error for the hidden layer units:  $\delta_j = f'(o_j) \sum_k w_{jk}\delta_k$ .
3. Update the weights for each layer using the gradient descent rule:

$$w_{ij} \leftarrow w_{ij} + \eta \cdot \delta_j \cdot o_i \quad (18)$$

where  $\eta$  is the learning rate and  $o_i$  is the activation of the input to unit  $j$ .



**Repeat:** Repeat the above steps for each training example until convergence or until a predefined number of iterations is reached. Convergence is achieved when the total error  $E(\vec{w})$  is sufficiently small or no longer decreases significantly.

This process allows the network to adjust its weights iteratively to minimize the error and improve its performance on the training data. While convergence to a global minimum cannot be guaranteed due to the potential for local minima, this algorithm provides a practical and effective approach for training multilayer neural networks.

---

**Algorithm 1** Backpropagation Algorithm

---

**Input:** Training examples  $\langle \vec{x}, \vec{t} \rangle$ , where  $\vec{x}$  is the input vector and  $\vec{t}$  is the target vector.

**Output:** Trained weights  $w_{ij}$  minimizing the error function.

**Initialize:** Randomly set all weights  $w_{ij}$  to small values. Set the learning rate  $\eta > 0$ .

**Forward Pass**

1. Present the input vector  $\vec{x}$  to the network.
2. For each neuron  $j$  in hidden and output layers, compute:

$$o_j = f \left( \sum_i w_{ij} x_{ij} \right)$$

where  $f(\cdot)$  is a differentiable activation function (e.g., sigmoid, ReLU).

**Backward Pass**

1. Compute the error signal  $\delta_k$  for each output unit  $k$ :

$$\delta_k = (t_k - o_k) f'(o_k)$$

2. Compute the error signal  $\delta_j$  for each hidden unit  $j$ :

$$\delta_j = f'(o_j) \sum_k w_{jk} \delta_k$$

3. Update the weights  $w_{ij}$  for each connection:

$$w_{ij} \leftarrow w_{ij} + \eta \cdot \delta_j \cdot o_i$$

Repeat Perform the *Forward Pass* and *Backward Pass* for all training examples.

Until Convergence is achieved (e.g., total error  $E(\vec{w})$  becomes sufficiently small or stops decreasing).

---

## 1.4 Universal Approximation Theorem

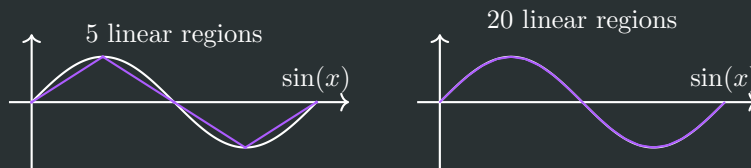


Figure 3: Visualization of the Universal Approximation Theorem with 5 and 20 linear regions

George Cybenko published "Approximation by superpositions of a sigmoidal function" proposing a theorem which shows that a finite sum of sigmoidal functions can densely approximate any continuous function

in  $C(I^n)$  where:

- $I^n$ :  $[0, 1]^n$  is the n-dimensional unit cube.
- $C(I^n)$ : space of a continuous functions on  $I^n$ , with the *supremum* norm  $\| \cdot \|$

Cybenko's theorem demonstrates that a neural network with sufficiency capacity, even if shallow, can accurately represent any continuous one-dimensional function within specified range. Usually, the Cybenko's theorem it's refereed as a case with arbitrary-width.

To easily understand *UAT*, we can visualize how it works in Figure ??